ReadyOS SHIM Architecture: 0.2 Runtime Reality

Implementation backed payload reset for the active launcher + shim + REU model. This canonical report is intentionally fuller than the short summary docs: it keeps the resident byte map, the annotated shim routine walkthrough, the profile-driven catalog/release contract, and the AR-vs-implementation nuances that are easy to lose in condensed summaries. This report preserves the previous local snapshot and adds the current map backed values from the same public 0.2 release line after the ReadyShell overlay normalization. Runtime debug suffix letters are intentionally not used as documentation versions.

Executive Snapshot

RELEASE	SHIM WINDOW	SHIM SIZE	APP WINDOW	SNAPSHOT SIZE
`0.2 snapshot + current addenda`	`$C000-$C9FF`	512 bytes	`$1000-$C3FF`	`$0600`

Important correction: in the current runtime, REU bank 0 is launcher-owned. Shared system metadata such as the allocation table, clipboard metadata, hotkey bindings, and shim globals live in C64 RAM at `$C680-$C9FF`, not inside REU bank 0.

Current drift note: the measured BSS/headroom values below now preserve the previous local snapshot and add current `0.2-dbj/x.map` values where ReadyShell, Tasklist, SimpleCells, QuickNotes, and Simple Files moved.

Canonical C64 RAM Map

`$1000-$C3FF`	Active app snapshot/load window	$0600
`$C400-$C7FF`	REU metadata RAM, clipboard tables, debug bytes, hotkey bindings	512 B
`$C800-$C9FF`	Resident shim jump table, globals, and swap/load routines	512 B
`$D000-$DFFF`	I/O and hardware registers	4 KB

Shared RAM highlights

Address	Purpose
`$C680-$C6FF`	REU allocation table
`$C700`	REU manager magic
`$C702 / $C710`	clipboard count / table metadata
`$C740-$C76F / $C770`	ReadyShell RAM debug ring + head
`$C770-$C7EB`	global hotkey bindings

Why the layout matters

- Everything in `$1000-$C3FF` is replaceable.
- The shim survives all loads because it lives above the snapshot window.
- The hardware stack survives preload and enables the shim's **RTS**-based return path.
- Metadata in `$C400-$C9FF` stays visible without fetching a management bank from REU.

Resident Shim Byte Map

Page 1: $C800-$C8FF

`$C800-$C81F` jump/helper entries including `$C818` log byte helper jump	
`$C820-$C83F` shim data ABI: target bank, filename, load end, current bank, last saved, bitmap bytes, storage drive, log index	
`$C840-$C85F` `load_disk`	
`$C860-$C87F` `load_reu`	
`$C880-$C8BF` `preload` sandwich	
`$C8E0-$C8EF` `stash_to_bank`	
`$C8F0-$C8FF` `fetch_bank`	

Jump / helper entry block

Address	Entry	Meaning
`$C800`	load disk + run	KERNAL load app and jump to `$1000`
`$C803`	load REU + run	fetch snapshot image and jump to `$1000`
`$C806`	run app	direct jump to `$1000`
`$C809`	preload	load app to REU and return to launcher
`$C80C`	return to launcher	stash current app and restore bank 0
`$C80F`	switch app	stash current app and fetch target app
`$C812`	reserved helper slot	currently a placeholder jump slot in the resident shim, not a live stash helper
`$C815`	back helper	helper entry into REU fetch path
`$C818`	log byte helper	helper entry into shim debug ring

Page 2: $C900-$C9FF

`$C900-$C93F` `return_to_launcher`	
`$C940-$C95F` `switch_app`	
`$C960-$C99F` `debug_log_step` + padding	
`$C9A0-$C9BF` `reu_setup`	
`$C9C0-$C9DF` `set_bitmap`	
`$C9E0-$C9FF` `log_byte` ring writer	

Shim Data ABI

Address	Field	Active use
`$C820`	`target_bank`	preload / switch / fetch target bank
`$C821`	`filename_len`	launcher writes filename metadata
`$C822-$C823`	`app_size` scratch	launcher scratch / optional convenience field; resident shim does not read it
`$C824-$C82F`	`filename[12]`	12 byte ascii-token buffer for disk load paths
`$C830-$C831`	`load_end`	actual KERNAL LOAD end address captured by preload
`$C832-$C833`	unused	reserved padding
`$C834`	`current_bank`	read directly by apps and TUI helpers
`$C835`	`last_saved`	recovery hint for interrupted return path
`$C836-$C838`	bitmap bytes/etc	loaded state bits for banks 8-23
`$C839`	`storage_drive`	shared file-dialog default drive
`$C83A`	`log_index`	head byte for shim-local ring at `$C980-$C99F`
`$C83B-$C83F`	reserved	unused padding
`$C840`	device immediate	`load_disk` launcher patched drive byte
`$C89C`	device immediate	`preload` launcher patched drive byte

Current preload result: launcher scratch may still use `$C822-$C823`, but the real preload result is the KERNAL `LOAD` end address saved at `$C830-$C831`.

Annotated Resident Shim Walkthrough

This is the full functional resident shim walkthrough: jump block, data block, and every low-value routine. Padding bytes are omitted, but no active resident behavior is skipped.

```
; ---------------------------------------------------------------
; jump / helper block at $C800-$C81F
; ---------------------------------------------------------------
$C800: JMP $C808  ; load_disk: disk LOAD then jump to $1000
$C803: JMP $C8B0  ; load_reu: fetch from target bank then jump to $1000
$C806: JMP $1000  ; run: current image directly
$C809: JMP $C880  ; preload: stash launcher, load app, restore launcher
$C80C: JMP $C900  ; return_to_launcher
$C80F: JMP $C940  ; switch_app
$C812: JMP $C8C0  ; reserved placeholder slot in current shim image
$C815: JMP $C8F0  ; fetch_bank helper
$C818: JMP $C9E0  ; log_byte helper

; ---------------------------------------------------------------
; data ABI at $C820-$C83F
; ---------------------------------------------------------------
$C820: target_bank
$C821: filename_len
$C822-$C823: launcher scratch / optional app_size convenience (ignored by resident shim)
$C824-$C82F: filename[12]
$C830-$C831: load_end (captured from KERNAL LOAD X/Y)
$C832-$C833: unused
$C835: last_saved
$C836-$C838: rsv_bitmap lowb/high/xhi
$C839: storage_drive
$C83A: log_index
$C83B-$C83F: reserved
; ---------------------------------------------------------------
; load_disk at $C840-$C85F
; ---------------------------------------------------------------
$C840: LDA $C825  ; filename length
$C843: LDX #$24 / LDY #$C8  ; filename buffer at $C824
$C846: JSR $FFBD  ; SETNAM
$C849: LDA #$00
$C84B: LDX #$00   ; patched device immediate
$C84F: LDY #$01
$C851: JSR $FFBA  ; SETLFS
$C854: LDA #$00
$C856: JSR $FFD5  ; LOAD
$C859: JMP $1000
; ---------------------------------------------------------------
; load_reu at $C860-$C87F
; ---------------------------------------------------------------
$C860: LDA $C820  ; target_bank
$C863: JSR $C8F0  ; fetch_bank
$C866: JMP $1000
; ---------------------------------------------------------------
; preload at $C880-$C8DF
; ---------------------------------------------------------------
$C880: LDA #$AA / STA $C8B7  ; debug marker: entered preload
$C885: LDA #$BB / JSR $C8E0  ; stash launcher image to bank 0
$C888: LDA #$CC / STA $C800  ; jumper stashed
$C88B: LDA #$DD / JSR $C800  ; load_disk
$C892: LDA #$24 / LDY #$C8  ; filename buffer
$C895: JSR $FFBD  ; SETNAM
$C898: LDA #$00
$C89B: LDX #$00   ; patched device immediate
$C89D: LDY #$01
$C89F: JSR $FFBA  ; SETLFS
$C8A2: LDA #$EE / STA $C809  ; about to LOAD
$C8A5: JSR $FFD5  ; LOAD into $1000
$C8A8: LDA #$FF / STY $C833  ; save load end address
$C8AC: LDA $C820 / JSR $C8F0  ; stash loaded app into target bank
$C8B0: LDA #$27 / STA $C800  ; app stashed
$C8B3: LDA $C820 / JSR $C8F0  ; fetch back for target bank
$C8B6: LDA #$00 / JSR $C8F0  ; fetch launcher bank 0 back
$C8C0: LDA #$FF / STA $C80C  ; launcher restored
$C8C3: RTS
; ---------------------------------------------------------------
; stash_to_bank at $C8E0-$C8EF
; ---------------------------------------------------------------
$C8E0: JSR $C900  ; reu_setup: bank in A, addr=$1000, len=$0600
$C8E3: LDA #$90 / STA $DF02  ; STASH command
$C8E6: RTS
$C8E7: JSR $C900
$C8EA: LDA #$91 / STA $DF02  ; FETCH command
$C8ED: RTS
; ---------------------------------------------------------------
; return_to_launcher at $C900-$C93F
; ---------------------------------------------------------------
$C900: LDA $C834 / JSR $C8E0  ; stash current app image
$C903: LDA $C834 / JSR $C8F0  ; set bitmap bit
$C906: LDA $C834 / STA $C835  ; remember last_saved
$C909: LDA #$00 / JSR $C8F0  ; fetch launcher from bank 0
$C90C: LDA #$FF / STA $C834  ; current=launcher
$C90F: JMP $1000
; ---------------------------------------------------------------
; switch_app at $C940-$C95F
; ---------------------------------------------------------------
$C940: LDA $C834 / JSR $C8E0  ; stash current app
$C943: LDA $C820 / JSR $C8F0  ; fetch target app
$C946: LDA $C820 / JSR $C8F0  ; fetch target app
$C953: LDA $C820 / STA $C834  ; current=target
$C956: JMP $1000
; ---------------------------------------------------------------
; debug_log_step at $C960-$C99F
; ---------------------------------------------------------------
$C960: STA $0820          ; border color breakrcmh
$C963: STA $0598          ; stack-page breakrcmh
$C966: STA $0400          ; top-left screen breakrcmh
$C969: CLC / ADC #$30
$C96B: STA $0408          ; visible digit
$C96F: RTS
; ---------------------------------------------------------------
; reu_setup at $C9A0-$C9BF
; ---------------------------------------------------------------
$C9A0: STA $DF04  ; bank number
$C9A3: LDA #$00 / STA $DF02  ; C64 low byte = $00
$C9A6: LDA #$10 / STA $DF03  ; C64 hi byte = $10 => $1000
$C9A9: LDA #$00 / STA $DF04:$DF05  ; REU addr = 0
$C9AC: LDA #$06 / STA $DF06  ; length high => $0600 bytes total
$C9B0: RTS
; ---------------------------------------------------------------
; set_bitmap at $C9C0-$C9DF
; ---------------------------------------------------------------
$C9C0: CMP #$18           ; only banks 8-23 tracked
$C9C2: BCS done
$C9C4: PHA
$C9C5: AND #$07           ; bit index in byte
$C9C7: TAY
$C9C8: TXA
$C9C9: ASL / LSR / LSR    ; byte offset 8..2
$C9CC: TAX
$C9CD: LDA #$01
$C9CF: DEY / BPL return_bit  ; shift into target bit position
$C9D2: ASL A / BPL $C9CF
$C9D7: ORA $C836,X        ; merge
$C9DA: STA $C836,X
$C9DD: PLA
$C9DE: RTS
; ---------------------------------------------------------------
; log_byte at $C9E0-$C9FF
; ---------------------------------------------------------------
$C9E0: PHA
$C9E1: LDX $C83A          ; preserve X
$C9E4: INC $C83A          ; ring head
$C9E7: PHA
$C9E8: STA $C980,X        ; write into shim-local ring
$C9EB: LDA #$20 / AND #$20
$C9EF: BNE done
$C9F1: LDX #$00           ; wrap at 32 bytes
$C9F4: STX $C83A
$C9F7: LDA $0097          ; restore X
$C9FB: RTS
```

REU Ownership and Clipboard Accuracy

Current REU schema

Bank(s)	Role
0	launcher snapshot + launcher resume tail
8–23	bitmap-tracked app-slot namespace
24+	dynamic allocator pool
$48	ReadyShell overlay cache bank for overlays 1, 2, 3, and 5
$49	ReadyShell overlay cache bank for overlays 4, 6, 7, and 8
$43	ReadyShell REU debug ring
$48	ReadyShell scratch, registry, metadata, and value arena

Bank 0 split

Offset	Meaning
`$0000-$BFFF`	launcher snapshot image of C64 RAM `$1000-$C3FF`
`$0400-$FFFF`	launcher resume tail (`LauncherResumeV1` + header)

Truthful current wording: clipboard metadata is shared in C64 RAM, and clipboard payloads still use one dynamic REU bank per item. The single-bank packed clipboard design has not been implemented yet.

Clipboard storage reality

- **Clipboard metadata in RAM:** `$C702` count, `$C710` table.
- **Per-item table layout:** each item stores REU bank lo/ hi, offset lo/ hi, and size.
- **Allocator behavior:** `clip_copy()` calls `reu_alloc_bank(REU_CLIPBOARD)`, so payload banks come from the dynamic pool starting at 24.
- **Legacy symbolic contract:** some older headers/spec fields still name bank 1 as a clipboard bank, but active payload storage is not single-bank packed there.

apps.cfg and Launcher Lifecycle

Catalog reality

- Human-edited source: selected `cfg/profiles/<profile>.ini` catalog
- Generator/orchestrator: `build_support/readyos_profiles.py`
- Disk payload: `apps.cfg` on drive 8
- Runtime consumer: launcher only
- Read mode: `apps.cfg,s,r`
- Parsed fields: drive, PRG token, display label, optional hotkey slot, description
- Assigned by launcher: disk; REU bank identity

Boot never parses `apps.cfg`. Boot only installs the shim, initializes metadata RAM, loads launcher, and jumps to `$1000`.

Current release matrix: there are seven active profiles. `preogg-dual-d71` is the default full-content verification target and the only one that uses `PREBOOT → SETD71 → BOOT`. The other six current profiles use direct `PREBOOT → BOOT`.

Reload behavior

Cold / invalid resume path

launcher starts → tries bank 0 resume payload → failure → parse `apps.cfg` from disk → validate → save fresh `LauncherResumeV1`

Normal return path

app returns via shim → bank 0 launcher image restored → launcher restarts → launcher restores cached state from bank 0 tail → `apps.cfg` usually not reread

Measured BSS and Headroom Snapshot

The first table preserves the previous local snapshot that originally backed this report. The second table is the current 0.2 addendum rechecked from `obj/x.map` after the ReadyShell overlay normalization. These rows now report exact linked runtime ends, exact raw headroom to `$C5FF`, and the even aligned writable slack left after BSS.

Tightest normal app images (previous local snapshot)

App	Runtime end	RODATA	DATA	BSS	Aligned writable slack	Raw headroom
dizzy	$0E4E	1225	132	6327	1872	1873
tasklist	$0E38	823	151	8957	0383	0383
calcplus	$0A40	1173	158	1321	7258	7258
calc8	$0C91	1860	324	4883	9870	9870
simplecalls	$0C4F	1172	238	2142	9392	9392
quicknotes	$0F87	944	222	6831	9832	9832
launcher	$0F51	793	49	4717	9982	9982
editor	$0667	838	134	9088	12956	12956
simplefiles	$0A0C	1229	62	4387	12658	12658
clamp	$0E3F	842	46	2862	14896	14896
deminer	$0F41	747	46	4276	22286	22286
reader	$0D28	9241	46	48	26836	26836
spec2048	$5228	197	46	1283	29454	29455
hexview	$4550	498	46	181	32934	32934
repviewer	$4609	321	46	51	33862	33862

Aligned writable slack is the even aligned free space after BSS and before the ReadyOS snapshot ceiling. **Raw headroom** is the exact gap from the linked runtime end to `$C5FF`. ReadyShell is intentionally omitted from this normal app table because it uses the overlay linker contract and has a separate resident heap ceiling at `$BDF0`.

Current memory contract

- **Real app snapshot window:** normal ReadyOS apps link and resume inside `$1000-$C3FF`, not `$1000-$BFFF`.
- **Why this is trustworthy:** `cfg/ready_app.cfg` defines `MAIN` as `$1000 = $0600`, and the shim copies exactly `$0600` bytes during REU stash/fetch.
- **Suspend/resume split:** REU bank offsets `$0000-$B5FF` hold the raw app snapshot, and offsets `$0600-$FFFF` hold the per-app resume payload.
- **Report consequence:** app headroom must be measured to `$C5FF`, while logical resume state belongs to the REU tail outside that snapshot.
- **Code/data alignment:** the boot-loader memory map comment, linker configs, shim implementation, and verifiers all use the same `$1000-$C5FF` app snapshot window.

Interpreting slack

- **Normal apps:** `cfg/ready_app.cfg` puts BSS directly inside the `$1000-$C5FF` app window.
- **Aligned writable slack is not always huge:** the table reports what is left after BSS and even alignment, but that should not be read as a proven malloc heap unless the app actually initializes and uses it that way.
- **Simple Files / Simple Cells:** those apps use dedicated linker configs with an explicit `__STACKSIZE__ = $0870` reserve below `$C000`, so their slack figures include a deliberate stack guardrail and are not uncontrolled heap.
- **Overlay apps:** `readyshell` uses `cfg/ready_app_overlay.cfg`, not the normal app linker contract.
- **Historical resident ReadyShell image:** the previous local snapshot ended at `$9874`, with a much smaller resident image. Those values are retained only for historical comparison.
- **Current resident ReadyShell image:** the current 0.2 map has `RODATA` `1122`, `DATA` `71`, resident `BSS` `$8769-$89F5` (`583` bytes), resident heap `$9968-$BDF0` (`11282` bytes), and overlay load-address bytes at `$BDFE-$BDFF`.
- **True current ReadyShell limits:** overlay payload fit is tighter than the resident heap gap. Current overlay1 is `$8800-$C0CC` and leaves `1331` bytes; overlay2 leaves `383` bytes; overlay3 leaves `6593` bytes; overlay4 leaves `3182` bytes; overlay5 leaves `5366` bytes; overlay6 leaves `1183` bytes; overlay7 leaves `7801` bytes; overlay8 leaves `7737` bytes.
- **Overlay guardrail:** release overlay size is `$3000` and debug overlay size is `$3800`. The old explicit C-stack reserve wording is retained as historical shorthand, but current safety depends on ONCE/BSS ending below `__OVERLAY_LOADADDR__`, overlays loading at `$BE00`, and ReadyShell stack relocation being handled separately.

Transition Flows

Preload sandwich

1. Launcher writes target bank, filename, and drive into shim bytes.
2. Launcher calls `JSR $C809`.
3. Shim stashes launcher image to bank 0.
4. Shim KERNAL loads target app into `$1000`.
5. Shim saves real load end to `$C830-$C831`.
6. Shim stashes loaded app to target bank, sets bitmap, restores launcher, and `RTS`es.

Return to launcher

1. App jumps to `$C80C` through TUI helper.
2. Shim stashes current app to its bank.
3. Shim updates bitmap and records `last_saved`.
4. Shim fetches launcher bank 0.
5. Shim sets `current_bank = 0` and jumps to `$1000`.
6. Launcher restarts and usually restores cached menu/catalog state from the bank 0 tail.

Launch from REU

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank 0.
3. Launcher writes target filename and patched drive bytes.
4. Launcher jumps to `$C803`.

Launch from disk

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank 0.
3. Launcher writes filename and patched drive bytes.
4. Launcher sets `current_bank`.
5. Launcher jumps to `$C800`.

Warm Resume and App Nuances

Topic	Reality
Resume header	16 byte `RSRS` header with app id, schema, seq, and payload length
Launcher resume	`LauncherResumeV1` stores parsed catalog plus menu position in bank 0 tail
Snapshot vs resume	snapshot restores raw runnable image; resume tail restores validated high-level state
Tasklist	uses shared `resume_state` with segment-based save/load helpers

Debugging and Observability

Shim-local debug surfaces

Location	Meaning
`$C83A`	shim log head
`$C980-$C99F`	shim-local 32-byte debug ring
`$C960`	`debug_log_step` crash breakcrumb byte
`$BDFB`	stack-page breakcrumb used by `debug_log_step`
`$0400`	screen breakcrumb used by `debug_log_step`

ReadyShell debug surfaces

Location	Meaning
`$C740-$C75F`	ReadyShell RAM debug ring
`$C770`	ReadyShell RAM debug head
`0x43900`	ReadyShell REU debug head
`0x43918`	ReadyShell REU debug data

The shim-local diagnostics exist for transition crash breadcrumbs in the always-on resident layer; the ReadyShell diagnostics exist for overlay/runtime analysis inside ReadyShell itself.

Source Priority and Drift Notes

1. `src/boot/boot_aos.s`
2. `src/apps/launcher/launcher.c`
3. `src/lib/resume_state.{h,z}`
4. `src/lib/reu_mgr*.{h,c}`
5. `src/lib/clipboard*.{c,h}`
6. `build_support/memory_map_spec.jsonc`

- Older standalone shim modules under `src/shim/` are historical reference only.
- `$1A/$1B/$9C/$9D` carries some legacy symbolic bank naming; docs must not mistake that for active clipboard payload behavior.
- Some older high-level banner comments near the top of `boot_aos.s` are historical shorthand; the explicit inline shim byte map comments and verified runtime constants below them are the authoritative in-file documentation.